

PROFILE SUMMARY

- Senior Python Developer with 8 years of experience building Python back-end services at consumer scale across visual discovery, ads ranking, and creator platforms, specializing in FastAPI services, asyncio concurrency, and SQLAlchemy 2.x data access.
- Hands-on coverage across Python (Python 3.12), web (FastAPI), ORM (SQLAlchemy 2.x, PostgreSQL), async (asyncio), and containers (Kubernetes, AWS) with strong fundamentals in PEP 8 discipline, full type hints, dataclasses, comprehensions, generators, and context-manager idioms.
- Deep expertise in async/await and asyncio orchestration, REST and GraphQL API design, Pydantic validation and dependency injection, and cProfile and py-spy performance tuning, applying methodologies such as test-driven development with pytest fixtures and parametrize and strict typing with mypy and ruff on every PR to deliver reliable, well-tested Python services that hold up at consumer-app scale.
- Engaged collaborator working cross-functionally with Product, Data Science, ML, and Platform teams in trunk-based and review-heavy product teams, contributing to API design forums, on-call rotations, and architecture reviews with an ownership-first mindset and clean handoffs.
- Mentor who shares technical excellence and fosters a culture of type-safe, test-covered Python and PEP-driven code quality through PR reviews and pattern docs, while running Python guild and API design review sessions and authoring widely used service and data-access templates.

TECHNICAL SKILLS

Languages & Runtime:	Python 3.12, Python 3.11, Python 3.10, type hints (PEP 484/604), dataclasses, generators, comprehensions, context managers, the GIL, asyncio
Web Frameworks & APIs:	FastAPI, Django, Flask, Litestar, Starlette, Pydantic, REST, GraphQL with Strawberry, OpenAPI, middleware, dependency injection
Async & Concurrency:	asyncio, async/await, anyio, trio, async generators, multiprocessing, threading, concurrent.futures, Celery, Dramatiq, httpx, aiohttp
Data Access & ORM:	SQLAlchemy 2.x, Django ORM, Tortoise ORM, SQLModel, Alembic, PostgreSQL, MySQL, SQLite, Redis, MongoDB, DynamoDB
Testing & Quality:	pytest, pytest-asyncio, fixtures, parametrize, unittest, hypothesis, responses, respx, Testcontainers, factory_boy, freezegun, coverage.py
Tooling & Packaging:	uv, Poetry, Hatch, pip-tools, ruff, Black, isort, mypy, pyright, pre-commit, tox, nox
Data, ML & Scripting:	pandas, polars, NumPy, PyArrow, scikit-learn, PyTorch, Hugging Face, Jupyter, ETL pipelines, Airflow, Prefect
Deployment & Cloud-Native:	Docker, multi-stage builds, distroless, Kubernetes, AWS Lambda, GCP Cloud Functions, Gunicorn, Uvicorn, Hypercorn, GitHub Actions, AWS

EDUCATION

University of Washington **B.S. in Computer Science**

Seattle, WA • Sep 2014 - Jun 2018

WORK EXPERIENCE

Pinterest **Senior Python Developer**

Seattle, WA • May 2021 - Present

- Owned idiomatic Python service development end to end on the creator and ads back-end platform serving 480M monthly users, shipping creator API, ads ingestion, and moderation pipeline across 14 type-hinted Python services held to PEP 8

and the Zen of Python.

- Tuned async and concurrency with **asyncio task groups with bounded semaphores and httpx clients**, structured cancellation on every request path, and multiprocessing for CPU-bound jobs that the GIL would otherwise pin, capping concurrent tasks at **1024** and pulling fan-out latency from **2.3s** down to **340ms** on the creator-feed aggregation path.
- Built backend and web services with **FastAPI with Pydantic models and dependency-injected services**, typed request and response schemas, middleware for auth and rate limiting, and OpenAPI specs generated from code across **180+** endpoints, dropping p95 from **220ms** to **48ms** on the hot creator-lookup path.
- Designed APIs and third-party integrations with **REST with httpx clients, webhook fan-out, and OpenAPI contract testing**, retries with exponential backoff, idempotency keys, and respX-mocked CI checks across **22** partner integrations, sustaining webhook throughput of **40k events/min** with zero duplicate-delivery incidents over the last four quarters.
- Owned data access and ORM work with **SQLAlchemy 2.x async sessions with Alembic migrations and Redis caching**, typed query builders, eager-loading audits, and prepared statements on hot reads, shipping **160+** schema migrations and cutting query p99 on the ads-attribution path by **71%**.
- Drove performance and profiling work with **cProfile and py-spy on hot paths with scalene memory profiling**, lru_cache and Redis layered caching, vectorized NumPy in inner loops, and Cython for the two hottest scoring kernels, dropping memory footprint per worker from **1.4GB** to **310MB** and cutting CPU cost on the ranking pipeline by **58%**.
- Shipped deployment and cloud-native Python with **multi-stage slim Docker images with Uvicorn workers on Kubernetes**, AWS Lambda for spiky webhook fan-out, Gunicorn-managed worker pools, and GitHub Actions pipelines on **Kubernetes** releasing **multiple times daily**, shrinking container images from **920MB** down to **148MB** across the fleet.

Zillow **Python Developer**

Seattle, WA • Jul 2018 - Apr 2021

- Drove testing and code quality with **pytest with fixtures, parametrize, and Testcontainers integration**, hypothesis property tests on parser code, respX-mocked HTTP boundaries, and coverage gates in CI, lifting test coverage from **44%** to **89%** and cutting production bugs per release by **64%**.
- Modernized Python tooling with **Poetry lockfiles, ruff, Black, and mypy on pre-commit**, tox matrices for 3.10 and 3.11, and pip-tools resolution gates on dependency PRs, dropping CI pipeline runtime from **18min** to **4min** and lifting mypy type coverage across the monorepo to **92%**.
- Built data and ML scripting workflows with **pandas and polars ETL jobs feeding scikit-learn pricing models**, PyArrow for columnar interchange, Airflow DAGs for nightly batches, and Jupyter notebooks converted to production scripts, cutting ETL runtime by **73%** while processing **180M** rows nightly across the home-valuation pipeline.
- Strengthened idiomatic Python foundations across the team through **dataclasses, generators, and context-manager refactors**, package-layout reviews, and weekly PEP reading sessions, while mentoring **7** engineers through pair-programming, code-review checklists, and a written Python idiom guide adopted across two squads.